A Tour of the Standard Library

Zhu Bicen

TD-LTE Cplane Support Team

November 8, 2011

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

Standard Library Facilities

- Basic run-time language support
- The C standard library
- Strings and I/O streams
- A framework of containers and algorithms using containers
- Support for numerical computation

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

C++ Style Output

```
typedef basic_ostream<char> ostream;
ostream cout;
template <class Ch, class Tr = char_traits<Ch> >
class basic_ostream: virtual public basic_ios<Ch,Tr
public:
    basic_ostream& operator<<(short n);
    basic_ostream& operator<<(int n);
    basic_ostream& operator<<(long n);
};
void f(int i){
     std::cout << "the value of i is " << i << '\n';
}
```

Theorem

For binary operator @, aa@bb can be interpreted as either aa.operator@(bb), or operator@(aa, bb).

String Stream

```
extern const char* std_message[];
string compose(int n, const string& cs)
{
  ostringsream ost;
  ost << "error(" << n << ")␣" << std_message[n]
      << "␣(user␣comment:␣" << cs << ')';
  return os.str();
}
```

LoggerStream

Example

```
class LoggerStream
{public:
  LoggerStream(...)
{
  //....
  m_stringStream << p_scName << "/"
                 << baseFilename(p_file)
                 << "#" << std::setw(3)
                 << std::left << p_id;
  //....
};
```

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

Initilization and Concatenation using non-member overloaded operators

```
1  void foo()
2  {
3     string imBlank;
4     string heyMom("Where␣are␣my␣socks?");
5     string standardReply = "Beamed␣into␣deep␣"
6          "space␣on␣wide␣angle␣dispersion?";
7     string useThisOneAgain(standardReply);
8  }
9
10 void f()
11 {
12    string s1 = "Hello"
13    string s2 = "World"
14    string s3 = s1 + ",␣" + "!\n";
15 }
```

Appending, inserting and concatenating strings

```
string big_news("I saw Elvis in a UFO. ");

cout << "Size = " << big_news.size() << endl;

cout << "Capacity = "
     << big_news.capacity() << endl;

// Insert this string in bigNews immediately
// before big_news[1]
big_news.insert(1, " thought I ");

// Add this to the end of the string
big_news.append("I've been working too hard.");
```

Manipulate Substrings

```
1 string name = "Niels␣Stroustrup";
2 void f()
3 {
4     string s = name.substr(6, 10);
5     name.replace(0, 5, "Nicholas");
6 }
```

Searching in strings

- find()
- find_first_of()
- find_last_of()
- find_first_not_of()
- find_last_not_of()
- rfind()

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

Build-in Array

```
1 struct Entry{
2   string name;
3   int number;
4 };
5 Entry phone_book[1000];
6 void print_entry(int i)
7 {
8     cout << phone_book[i];
9 }
```

However, a built-in array has a fixed size. If we choose a large size, we waste space; if we choose a smaller size, the array will overflow. In either case, we will have to write low-level memory management code.

Vector takes care of Memory Management

```
vector<Entry> phone_book(1000);
void print_entry(int i)// simple use, exactly as fo
{
  cout << phone_book[i].name << ' '
       << phone_book[i].number << '\n';
}
void add_entries(int n)//increase size by n
{
  phone_book.resize(phone_book.size() + n);
}

vector<Entry> book(1000); //vector of 1000 elements
vector<Entry> books[1000]; // 1000 empty vectors
```

Constructor

```
vector<Record> vr(10000);
void f(int s1, int s2)
{
  vector<int> vi(s1);
  vector<double>* p = new vector<double>(s2);
}
```

Elements of a vector allocated this way are initialized by the default constructor for the element type.

Constructor

```
class Num{ //
public:
    Num(long);
    //no default constructor
    //...
};
vector<Num> v1(1000);        //error: no default Num
vector<Num> v2(1000, Num(0));
```

If a type does not have a default constructor, it is not possible to create a vector with elements of that type without explicitly providing the value of each element.

The copy constructor and the copy-assignment operators copy the elements of vector

```
1  void f1(vector<int>&);          //common style
2  void f2(const vector<int>&);  //common style
3  void f3(vector<int>);           //rare style
4
5  void h()
6  {
7      vector<int> v(10000);
8      //....
9      f1(v);   //pass a reference
10     f2(v);   //pass a reference
11     f3(v);   //copy the 10000 elementinto a new vecto
12 }
```

When a vector holds many elements, such innocent-looking assignments and initializations can be prohibitively expensive.

Stack operation

```
1
2 void f(vector<char>& s)
3 {
4   s.push_back('a');
5   s.push_back('b');
6   s.push_back('c');
7   s.pop_back();
8   if(s.[s.size() -1] != 'b') error("Impossible");
9   if(s.back() != "a") error("should␣never␣happen");
10 }
```

List Operation

```
vector<string> fruit = {"peach","apple",
  "kiwifruit", "pear",
  "starfruit", "grape"};

sort(fruit.begin(), fruit.end();
auto p1 = find_if(fruit.begin(), fruit.end(),
                  initial('p');
auto p2 = find_if(p1, fruit.end(), initial('p');
fruit.erase(p1, p2.base());
```

Addressing Elements

```
1  template<class C> void f(C& c)
2  {
3     c.erase(c.begin() +7);                    //ok
4     c.erase(&c[7]);                           //not general
5     c.erase(c+7);                             //error
6     c.erase(c.back());                        //error:c.back()
7     c.erase(c.end() -2);                      //ok(second to l
8     c.erase(c.rbegin() +2);                   //error
9     //vector::reverse_iterator and vector::iterator
10    //are different types
11    c.erase((c.rbegin() +2).base());
```

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

A List is a Sequence

```
1 void print_entry(const string& s)
2 {
3     typedef list<Entry>::const_iterator LI;
4     for(LI i = phone_book.begin(); i != phone_book.e
5       Entry& e = *i; //reference used as shorthand
6       if( s == e.name)
7           cout << e.number;
8     }
9 }
```

Adding elements to a list t is easy

```
1 void add_entry(Entry& e, list<Entry>::iterator i)
2 {
3     phone_book.push_front(e);
4     phone_book.push_back(e);
5     phone_book.insert(i, e); // add before the eleme
6 }
```

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

Iteration Over a Map

```
1 void f(map<string, number>& phone_book)
2 {
3   typedef map<string, number>::const_iterator CI;
4   for(CI p = phone_book.begin();
5       p != phone_book.end(); ++p){
6     cout << p->first << '\t' << p->second << '\n';
7 }
```

Subscripting

```
mapped_type& operator[](const key_type& k);

void f()
{
  map<string,int> m;  // map starting out empty
  // create new entry for "Henry", initialize to 0,
  int x = m["Henry"];
  // create new entry for "Harry", initialize to 0,
  m["Harry"] = 7;
  // return the value from "Henry"  s  entry
  int y = m["Henry"];
  // change the value from "Harry"  s  entry to 9
  m["Harry"] = 9
}
```

Constructor

```
1 map<string, int> m1;
2 //specify comparison type
3 map<string, int, Nocase> m2;
4 map<string, int String_cmp> m3;
5 //pass comparison object
6 map<string, int> m4<String_cmp(literary));
```

Standard Containers

Container	Summary
vector	A variable-sized vector
list	A double-linked list
queue	A queue
stack	A stack
deque	A double-ended queue
priority_queue	A queue sorted by value
set	A set
multiset	A set in which a value can occur many times
map	An associative array
multimap	A map in which a key can occur many times

common algorithms for containers

```
1 void f(vector& ve, list<Entry>& le)
2 {
3     sort(ve.begin(), ve.end());
4     unique_copy(ve.begin(), ve.end(), le.begin());
5 }
6
7 unique_copy(ve.begin(), ve.end(), back_inserter(le)
```

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

Use of Iterator

```
1  int count(const string& s, char c)
2  {
3      int n = 0;
4      string::const_iterator i =
5          find(s.begin(), s.end(), c);
6      while( i != s.end(){
7          ++n;
8          i = find(i + 1, end(), c);
9      }
10     return n;
11 }
12 void f()
13 {
14    string m = "Mary␣had␣a␣little␣lamb";
15    int a_count = count (m, 'a');
16 }
```

Type of Iterator

any object that obeys a few simple rules like these is an iterator .
Furthermore, users rarely need to know the type of a specific iterator; each container "knows" its iterator types and makes them available under the conventional names i it te er ra a- - iterator and const_iterator.
For example, is the general iterator type for I rarely have to worry about the details of how that type is defined.

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

A predicate is used to control the algorithm

```
bool gt_42(const pair<const string, int>& r)
{
    return r.second > 42;
}

void f(map<string, int>& m)
{
  typedef map<string, int>::const_iterator MI;
  MI i = find_if(m.begin(), m.end(), gt_42);
}
void g(const map<string, int>& m)
{
  int c42 = count_if(m.begin(), m.end(), gt_42);
}
```

In ¡functional¿, the standard library supplies a few common predicates

Functor

```
1  template<class T> class Sum{
2      T res;
3  public:
4      Sum(T i = 0): res(i){}
5      void operator()(T x){res += x;}
6      T result const {return res;}
7  };
8  void f(list<double>& ld)
9  {
10     Sum<double> s;
11     for_each(ld.begin(), ld.end(), s);
12     cout << "the sum is " << s.result;
13 }
```

Algorithms Using Member Functions

```
1 void g(list<Shape*>& sh)
2 {
3     for_each(sh.begin(), sh.end(),
4                 mem_fun(&Shape::draw));
5 }
```

Algorithms Using Member Functions

```
class status {
  std::string name_;
  bool ok_;
public:
  status(const std::string& name):
      name_(name),ok_(true) {}
  void break_it() {
    ok_=false;
  }
  bool is_broken() const {
    return ok_;
  }
  void report() const {
    std::cout << name_ << " is " <<
      (ok_ ? "working nominally":"terribly broken")
  }
```

Algorithms Using Member Functions

```
std::vector<status> statuses;
statuses.push_back(status("status␣1"));
statuses.push_back(status("status␣2"));
statuses.push_back(status("status␣3"));
statuses.push_back(status("status␣4"));
statuses[1].break_it();
statuses[2].break_it();
for (std::vector<status>::iterator it=statuses.begi
  it!=statuses.end();++it) {
  it->report();
}
```

Algorithms Using Member Functions

```
1 std::for_each(
2   statuses.begin(),
3   statuses.end(),
4   std::mem_fun_ref(&status::report));
5
6 std::for_each(
7   statuses.begin(),
8   statuses.end(),
9   boost::bind(&status::report,_1));
```

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

Selected Standard Algorithms

Algorithm	Introduction
for_each()	Invoke function for each element (18.5.1)
find()	Find first occurrence of arguments (18.5.2)
find_if()	Find first match of predicate (18.5.2)
count()	Count occurrences of element (18.5.3)
count_if()	Count matches of predicate (18.5.3)
replace()	Replace element with new value (18.6.4)
replace_if()	Replace element that matches predicate with new value (18
copy()	Copy elements (18.6.1)
unique_copy()	Copy elements that are not duplicates (18.6.1)
sort()	Sort elements (18.7.1)
equal_range()	Find all elements with equivalent values (18.7.2)
merge()	Merge sorted sequences (18.7.3)

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

No Need to Manually Delete

```
1  int main() {
2    {
3    boost::scoped_ptr<std::string>
4    p(new std::string("Use␣scoped_ptr␣often."));
5
6    // Print the value of the string
7    if (p) std::cout << *p << '\n';
8
9    // Get the size of the string
10   size_t i=p->size();
11
12   // Assign a new value to the string
13   *p="Acts␣just␣like␣a␣pointer";
14
15   } // p is destroyed here, and deletes the std::st
16 }
```

Almost Like auto_ptr

```
1  void scoped_vs_auto() {
2
3    scoped_ptr<string> p_scoped(new string("Hello"));
4    auto_ptr<string> p_auto(new string("Hello"));
5
6    p_scoped->size();
7    p_auto->size();
8
9    scoped_ptr<string> p_another_scoped=p_scoped;
10   auto_ptr<string> p_another_auto=p_auto;
11
12   p_another_auto->size();
13   (*p_auto).size();
14 }
```

scoped_ptr Is Not the Same As const auto_ptr

```
1 const auto_ptr<A> no_transfer_of_ownership(new A);
```

scoped_ptr can be reset, but not const auto_ptr If you want to say that a resource is scoped, and that there's no way you'll relinquish ownership of it, spell it boost::scoped_ptr.

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

shared_ptr Usage

```
Foo * p = new Foo;

{//thread 1; executes concurrently with thread 2
 std::tr1::shared_ptr<Foo> pfoo1(p);
 pfoo1->func();
 return 0;
}
{//thread 2; executes concurrently with thread 1
 std::tr1::shared_ptr<Foo> pfoo2(p);
 pfoo1->another_func();
 return 0;
}
```

shared_ptr Construction

```
std::tr1:shared_ptr <void> pf(new Foo);

//API functions
DeviceContext * get_dc();
int release_dc();
void draw()
{
 //provide a user-defined deleter
 shared_ptr<DeviceContext> dc(get_dc(), release_dc)
 //...use dc
 //instead of deleting the dc pointer,
 //the release_dc() API function
 //will be called when dc is destroyed.
}
```

use_count() and unique()

The use_count() member function returns the number of clients currently sharing the same resource. The common:

```
use_count() == 1
```

query has its own name: unique():

```
shared_ptr<DeviceContext> dc(get_dc(), release_dc);
if(dc.unique())
{
 //...resource isn't shared with others
}
```

shared_ptr and Standard Library Containers

```
1 std::vector<shared_ptr<status> > s_statuses;
2 s_statuses.push_back(
3   shared_ptr<status>(new status("status␣1")));
4 s_statuses.push_back(
5  shared_ptr<status>(new status("status␣2")));
6 s_statuses[1]->break_it();
7 s_statuses[2]->break_it();
```

cyclic dependency

weak_ptr "weak reference" to an object that's already managed by a shared_ptr

click for More info about weak_ptr

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

Boost bind usage

```
1 #include <iostream>
2 #include "boost/bind.hpp"
3 void nine_arguments(
4   int i1,int i2,int i3) {
5   std::cout << i1 << i2 << i3 << '\n';
6 }
7 int main() {
8   int i1 = 1, i2 = 2, i3 = 3;
9   (boost::bind(&nine_arguments,_2,_1,_3))
10                 (i1,i2,i3);
11 }
```

Bind Usage

```
vector<LnAdjlToServedCellMapping> p_mappingList;
const SAdjServedCellInformation *p_serverCellList,

bool servedCellCmp(
  const SAdjServedCellInformation *p_serverCell1,
  const LnAdjlToServedCellMapping &p_serverCell2);

find_if(
  p_mappingList.begin(), p_mappingList.end(),
  boost::bind( servedCellCmp,
               &p_serverCellList[l_i], _1)) ;
```

Bind Usage

```
1 std::find_if(
2     l_searchResult.begin(), l_searchResult.end(),
3     boost::bind(&Utils::lnAdjlEcgiEqual,
4                 _1, p_source->cell_info.adjECGI));
```

Outline

1. Strings and I/O streams
   - I/O Streams
   - String
2. Containers
   - vector
   - list
   - map
3. Algorithms
   - Iterator
   - Traversals and Predicates
   - Standard Library Algorithms
4. Boost –Beyond STL
   - scoped_ptr
   - shared_ptr
   - boost.bind
   - boost.lambda

Boost Version Lambda

```
1  #include "boost/lambda/lambda.hpp"
2  #include "boost/function.hpp"
3  int main() {
4    using namespace boost::lambda;
5    (std::cout << _1 << " "
6               << _3 << " " << _2
7               << "!\n")
8      ("Hello","friend","my");
9
10   boost::function<void(int,int,int)> f=
11     std::cout << _1 << "*" << _2 << "+" << _3
12       << "=" <<_1*_2+_3 << "\n";
13   f(1,2,3);
14   f(3,2,1);
15 }
```

boost.lambda

C++11 Version Lambda

```
[capture](parameters)->return_type {body}
int main()
{
    char s[]="Hello World!";
    int Uppercase = 0; //modified by the lambda
    for_each(s, s+sizeof(s),
            [&Uppercase] (char c) {
              if (isupper(c))
              Uppercase++;
            });
    cout << Uppercase << " uppercase letters in: "
       << s <<endl;
}
```

Maybe the Simplest Lambda

```
1 #include "boost/lambda/lambda.hpp"
2 int main() {
3   using namespace boost::lambda;
4   std::vector<int> vec(3);
5   vec[0]=1;
6   vec[1]=2;
7   vec[2]=3;
8   std::for_each(vec.begin(),vec.end(),_1+=10);
9   std::for_each(vec.begin(),vec.end(),_1-=10);
10  std::for_each(vec.begin(),vec.end(),_1*=3);
11  std::for_each(vec.begin(),vec.end(),_1/=2);
12  std::for_each(vec.begin(),vec.end(),_1%=3);
13 }
```

lambda example

```
m_fsm->connectionReleaseAll(getTraceType(),
 &boost::lambda::_1->*&ConnectionInfo_t
                          ::ext3rdPartyConnection);
```

Thanks!